Ansible Automation - Use Cases

A reference of the use cases by which Ansible Automation Platform is commonly implemented, by domain. Each use case is described in the same format (Description, Collection, Key content, Notes) and mapped to Red Hat Certified and Validated content.

5 solution use cases · 7 product use cases · generated 2026-10-21 · community reference, not an official Red Hat publication

Contents

1. Kubernetes & OpenShift
    - Solution use cases (5)
        - Deploy applications to Kubernetes and OpenShift
        - Onboard OpenShift projects and quotas
        - Configure OpenShift cluster Day 2 settings
        - Back up Kubernetes applications
        - Run Kubernetes node maintenance and cluster housekeeping
    - Product use cases
        - Kubernetes (4)
        - Red Hat OpenShift (2)
        - Trilio for Kubernetes (1)

Kubernetes & OpenShift

Day 2 automation for OpenShift and Kubernetes clusters and applications.

Solution use cases

Use Case: Deploy applications to Kubernetes and OpenShift

Description	Deploy and update applications on Kubernetes or OpenShift from a single, auditable job template. The job applies manifests, Kustomize overlays or Helm charts, waits for the rollout to become healthy and rolls back if it does not. Environment differences are variables, so the same content promotes through development, test and production clusters.
Collection	`kubernetes.core` (Certified), `redhat.openshift` (Certified)
Key content	• `kubernetes.core.k8s` — Query the K8s API • `kubernetes.core.helm` — Manages Kubernetes packages with the Helm package manager • `kubernetes.core.helm_repository` — Manage Helm repositories. • `kubernetes.core.kustomize` — Build a set of kubernetes resources using a 'kustomization.yaml' file. • `kubernetes.core.k8s_info` — Describe Kubernetes (K8s) objects • `kubernetes.core.k8s_rollback` — Rollback Kubernetes (K8S) Deployments and DaemonSets • `redhat.openshift.openshift_process` — Process an OpenShift template.openshift.io/v1 Template • `redhat.openshift.openshift_route` — Expose a Service as an OpenShift Route.
Notes	• Use `kubernetes.core.k8s_info` with `wait` conditions rather than fixed sleeps to keep jobs fast and reliable. • Keep secrets out of Git. Retrieve them at run time from a vault or Automation Platform credentials. • If an in-cluster GitOps tool already reconciles the cluster, use Ansible for the surrounding steps (namespace setup, secrets, approvals) rather than competing with it. • Pair with Onboard OpenShift projects and quotas for the namespace preparation and Back up Kubernetes applications for protection. • Event-Driven Ansible: a Git or registry webhook can trigger the deployment job when a new image or chart version is published.

Use Case: Onboard OpenShift projects and quotas

Description	Give new application teams a ready-to-use OpenShift project in minutes: namespace, resource quota, default limits, network isolation, role bindings and group membership are all created from one request. Doing this with Ansible keeps every project to the same standard and leaves a record of who asked for what.
Collection	`kubernetes.core` (Certified), `redhat.openshift` (Certified), `infra.automation_development_office` (Validated)

Key content	• `kubernetes.core.k8s` — Query the K8s API • `kubernetes.core.k8s_info` — Describe Kubernetes (K8s) objects • `redhat.openshift.openshift_adm_groups_sync` — Sync OpenShift Groups with records from an external provider. • `redhat.openshift.openshift_adm_prune_auth` — Removes references to the specified roles, clusterroles, users, and groups • `infra.automation_development_office.ocp_namespace` — An collection. • `infra.automation_development_office.ocp_service_accounts` — An collection. • `infra.automation_development_office.ocp_network_policy` — Manage OpenShift network security policies — NetworkPolicy, AdminNetworkPolicy, BaselineAdminNetworkPolicy, MultiNetworkPolicy, and EgressFirewall.
Notes	• Quota changes are a common follow-up request; reuse the same job template with a new tier value. • Combine with Configure OpenShift cluster Day 2 settings so that identity providers exist before groups are synchronised. • The `infra.automation_development_office` roles are validated content from Red Hat consulting; review their variables before using them in production. • Event-Driven Ansible can trigger the same job from a ServiceNow or Git event when a project request is approved.

Use Case: Configure OpenShift cluster Day 2 settings

Description	After a cluster is installed, every cluster still needs the same set of settings: identity providers, certificates, operators, node networking, load balancing and logging. Capture them as Ansible content in Git and apply them to every cluster, so new clusters are brought to standard quickly and existing clusters can be checked for drift.
Collection	`kubernetes.core` (Certified), `redhat.openshift` (Certified), `infra.automation_development_office` (Validated)
Key content	• `kubernetes.core.k8s` — Query the K8s API • `kubernetes.core.k8s_cluster_info` — Describe Kubernetes (K8s) cluster, APIs available and their respective versions • `infra.automation_development_office.ocp_ldap_auth` — An collection. • `infra.automation_development_office.ocp_oidc_auth` — An collection. • `infra.automation_development_office.ocp_cert_manager` — An collection. • `infra.automation_development_office.ocp_operator_subscription` — An collection. • `infra.automation_development_office.ocp_nmstate_install` — Install Kubernetes NMState Operator, create NMState instance, and apply NodeNetworkConfigurationPolicy resources for physical networking. • `infra.automation_development_office.ocp_metallb_install` — Install MetalLB Operator, create MetalLB instance, and configure IPAddressPools, L2Advertisements, BGPPeers, and BGPAdvertisements. • `infra.automation_development_office.ocp_logging` — An Openshift Logging collection

Notes	• Cluster installation itself and the Red Hat Advanced Cluster Security configuration are outside this use case; see the Security use cases for RHACS. • The `infra.automation_development_office` roles are validated content; their descriptions vary in detail, so read each role's documentation and defaults first. • Use Onboard OpenShift projects and quotas after identity and operators are in place. • Event-Driven Ansible can trigger a drift correction job when a configuration object such as an operator subscription is changed outside Git.

Use Case: Back up Kubernetes applications

Description	Protect Kubernetes and OpenShift applications with application-consistent backups of their objects and persistent data. Automation Platform schedules the backups, verifies them, runs test restores and can move an application to another cluster, replacing ad-hoc scripts with a tested recovery process.
Collection	`trilio.trilio_kubernetes` (Certified), `kubernetes.core` (Certified), `infra.automation_development_office` (Validated)
Key content	• `trilio.trilio_kubernetes.trilio_kubernetes` — Ansible role to orchestrate backup, recovery and migration of Kubernetes applications and data using Trilio. • `kubernetes.core.k8s` — Query the K8s API • `kubernetes.core.k8s_info` — Describe Kubernetes (K8s) objects • `infra.automation_development_office.ocp_oadp` — An collection.
Notes	• Backup of VMs on OpenShift Virtualization is covered in Operate VMs on OpenShift Virtualization. • Storage-level snapshots and replication can complement application backups; see Manage snapshots, clones and replication. • Back up the cluster configuration, such as the manifests applied in Configure OpenShift cluster Day 2 settings, by keeping them in Git. • Event-Driven Ansible can start an on-demand backup when a change request is approved or a maintenance window opens.

Use Case: Run Kubernetes node maintenance and cluster housekeeping

Description	Take cluster nodes out of service for patching, hardware work or replacement without disrupting applications, and keep the cluster tidy by pruning old images, builds and unused references. The same jobs run on a schedule or inside larger patching and firmware workflows.
Collection	`kubernetes.core` (Certified), `redhat.openshift` (Certified)
Key content	• `kubernetes.core.k8s_drain` — Drain, Cordon, or Uncordon node in k8s cluster • `kubernetes.core.k8s_taint` — Taint a node in a Kubernetes/OpenShift cluster • `kubernetes.core.k8s_info` — Describe Kubernetes (K8s) objects • `kubernetes.core.k8s_cluster_info` — Describe Kubernetes (K8s) cluster, APIs available and their respective versions • `redhat.openshift.openshift_adm_prune_images` — Remove unreferenced images • `redhat.openshift.openshift_adm_prune_builds` — Prune old completed and failed builds • `redhat.openshift.openshift_registry_info` — Display information about the integrated registry.

Notes	• On OpenShift, node operating system updates are normally managed by the platform itself; use this use case for the surrounding drain and hardware steps, or for non-OpenShift Kubernetes clusters. • Pair with Update server firmware and BIOS for bare-metal nodes. • `kubernetes.core.k8s_exec`, `k8s_log` and `k8s_cp` help with troubleshooting steps inside the same workflow. • Event-Driven Ansible can react to node-not-ready events and start a diagnostic job.

Product use cases

Grouped from the modules of each certified or validated collection. Key content lists the components each use case supports, with the modules that manage them (create, update, delete) and query them (_info).

Kubernetes `kubernetes.core`

Certified Red Hat · version 6.6.0 · updated 2026-09-17 · 21 modules → 4 use cases

Use Case: Manage Kubernetes objects

Description	Create, patch, scale, roll back and inspect Kubernetes objects and services, and query the API.
Collection	`kubernetes.core` (Certified, Red Hat)
Key content	**Kubernetes objects** `k8s`, `k8s_info` **JSON patches** `k8s_json_patch` **Scaling** `k8s_scale` **Rollbacks** `k8s_rollback` **Services** `k8s_service` **Cluster and API information** `k8s_cluster_info` **Kubernetes API lookup** lookup `k8s` **Kustomize builds** lookup `kustomize` **ConfigMap and Secret names** filter `k8s_config_resource_name`

Use Case: Deploy applications with Helm

Description	Install and manage Helm releases, repositories, plugins and registry credentials, pull charts and render templates.
Collection	`kubernetes.core` (Certified, Red Hat)
Key content	**Helm releases** `helm`, `helm_info` **Helm repositories** `helm_repository` **Helm plugins** `helm_plugin`, `helm_plugin_info` **Chart downloads** `helm_pull` **Registry authentication** `helm_registry_auth` **Chart templates** `helm_template`

Use Case: Operate nodes and pods

Description	Drain, cordon and taint nodes, run commands in pods, copy files and fetch logs.
Collection	`kubernetes.core` (Certified, Red Hat)
Key content	**Node drain and cordon** `k8s_drain` **Node taints** `k8s_taint` **Pod commands** `k8s_exec` **Pod file copy** `k8s_cp` **Logs** `k8s_log` **kubectl connection** connection `kubectl`

Use Case: Connect to clusters

Description	Generate and update kubeconfig files.
Collection	`kubernetes.core` (Certified, Red Hat)
Key content	**Kubeconfig files** `kubeconfig`

Red Hat OpenShift `redhat.openshift`

Certified Red Hat · version 6.0.0 · updated 2026-08-17 · 13 modules → 2 use cases

Use Case: Deploy and expose applications

Description	Manage OpenShift objects, process templates, start builds, import images and expose services as routes.
Collection	`redhat.openshift` (Certified, Red Hat)
Key content	**OpenShift objects** `k8s` **Templates** `openshift_process` **Builds** `openshift_build` **Image imports** `openshift_import_image` **Routes** `openshift_route` **Integrated registry information** `openshift_registry_info` **oc connection** connection `oc`

Use Case: Administer clusters and clean up resources

Description	Authenticate to clusters, sync groups from external providers, migrate template instances and prune builds, deployments, images and authorization references.
Collection	`redhat.openshift` (Certified, Red Hat)

Key content	**Cluster login** `openshift_auth` **Group sync** `openshift_adm_groups_sync` **Template instance migration** `openshift_adm_migrate_template_instances` **Authorization pruning** `openshift_adm_prune_auth` **Build pruning** `openshift_adm_prune_builds` **Deployment pruning** `openshift_adm_prune_deployments` **Image pruning** `openshift_adm_prune_images`

Trilio for Kubernetes `trilio.trilio_kubernetes`

Certified Trilio · version 1.5.0 · updated 2026-02-24 · 0 modules → 1 use cases

Use Case: Back up, recover and migrate Kubernetes applications

Description	Orchestrate backup, recovery and migration of Kubernetes applications and their data with Trilio, using the role or the utility playbooks.
Collection	`trilio.trilio_kubernetes` (Certified, Trilio)
Key content	**Trilio for Kubernetes role** role `trilio_kubernetes` **Trilio utility playbook** playbook `trilio-utility` **Trilio utility playbook (local)** playbook `trilio-utility-local`